Design Trade-Offs and Challenges

1. Using App Runner vs. ECS or Lambda for Compute

App Runner provides simplified deployment, autoscaling, and HTTPS, reducing operational burden. The trade-off is reduced infrastructure control and higher costs for long-running workloads. It prioritizes developer speed over deep customization.

2. DynamoDB + RDS Combination

Using DynamoDB for catalog data and RDS PostgreSQL for transactional workloads improves performance and reliability. The trade-off is increased operational complexity, duplicated data models, and the need for careful consistency management.

3. CloudFront + Amplify Hosting

Amplify Hosting simplifies frontend deployment, while CloudFront provides global caching. The challenge is managing cache invalidation so users do not see outdated content. This requires deliberate cache behaviors and versioning strategies.

4. High Availability vs. Cost

Multi-AZ RDS, autoscaling compute, global CDN distribution, and caching layers improve uptime and performance, but they significantly increase cost. Teams must balance reliability with budget constraints.

5. Using Redis (ElastiCache) for Performance

ElastiCache provides very fast response times for sessions, carts, and hot data. Challenges include failover complexity, in-memory volatility, and the need for fallback strategies if Redis becomes unavailable.

6. Introducing SQS and Lambda Workers

SQS and Lambda provide reliable asynchronous processing. The trade-off is additional moving parts (queues, DLQs, retry logic). Teams must ensure idempotency and message integrity to avoid duplicate order processing.

7. Security Layers (WAF, IAM, Shield, KMS)

Security services strengthen the platform but increase configuration overhead. IAM misconfigurations, WAF rule tuning, and KMS API costs require continuous monitoring and expertise.

8. Complexity vs. Simplicity

This architecture is highly scalable and robust but introduces many interconnected services. Without strong logging, monitoring, and documentation, troubleshooting becomes difficult for small teams.